TRIBHUVAN UNIVERSITY

Institute of Science and Technology

Central Department of Computer Science and Information Technology

Selective Adoption of MobileNetV2 Architectural Principles for an Efficient Custom CNN in Plant Disease Classification

A Seminar Presentation

Presented by

Kriti Mandal

MSc.CSIT, 1st Semester

August 2026

Outline

- Introduction
- Problem Statement
- Objectives
- Literature Review
- Methodology
- Dataset and Preprocessing
- Model Architectures
- Convolutional Neural Networks
- Algorithm Details
- Batch Normalization and Global Average Pooling
- Evaluation Metrics
- Implementation Details
- Implementation Code
- Results
- Ablation Results
- Confusion Matrix (three slides)
- Per-Class Analysis
- Efficiency Analysis
- Conclusion
- Future Work

Introduction

- Plant diseases are a major cause of agricultural yield loss.
- Traditional identification depends on visual inspection by experts, which is time-consuming and prone to human error.
- Convolutional Neural Networks are widely used to identify plant diseases from leaf images and have obtained high classification accuracy.
- A model intended for field use should also be small and fast, since farmers may need to run it on a mobile phone without an internet connection.
- Lightweight architectures such as MobileNet and MobileNetV2 were developed for mobile and embedded applications.
- This study applies selected MobileNetV2 principles to a custom CNN and evaluates each one individually.

Problem Statement

- Accurate and timely detection of plant diseases from leaf images under real field conditions is difficult.
- Delay or inaccuracy in detection allows infections to spread quickly.
- This results in lower crop yields and significant financial losses for farmers.
- Deep learning models that achieve high accuracy are often large and slow, which limits their use on devices with limited computational resources.
- Existing studies mostly evaluate complete architectures, so the contribution of each individual architectural component is not known.

Objectives

- To evaluate the individual effects of global average pooling, batch normalization, depthwise separable convolutions and inverted residual blocks on classification performance and efficiency.

- To compare the CNN models with a pretrained MobileNetV2 reference model in terms of classification performance, model size and inference speed.

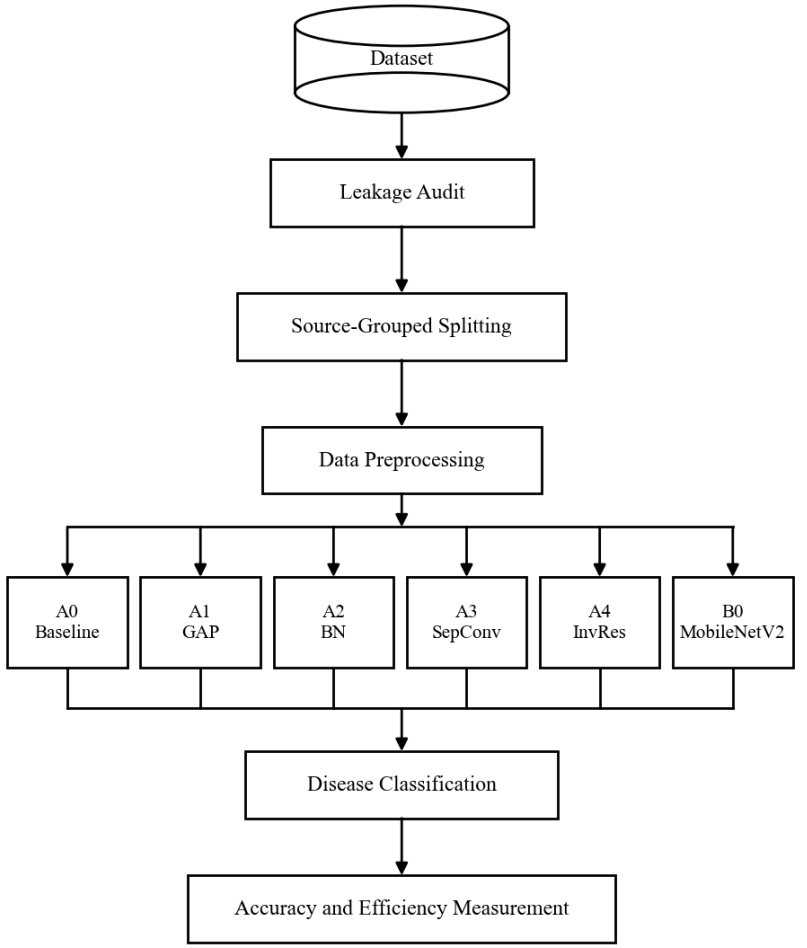
- To identify whether selective adoption of MobileNetV2 architectural principles can provide an effective and efficient alternative for plant disease classification.

Literature Review

- Early work on the PlantVillage dataset showed that deep CNNs can learn visual features from leaf images and classify a wide range of plant diseases, although the images were captured under controlled conditions.
- Later studies evaluated several deep learning architectures for disease detection and reported strong performance, but greater model complexity also increases computational and memory requirements.
- Custom CNNs have also been explored. A nine-layer network trained from scratch on an augmented dataset achieved high classification performance, showing that a model can work without depending on pretrained networks.
- Transfer learning has been widely used, since features learned from large image datasets reduce the training required, although pretrained models may contain components that are unnecessary for a particular task.
- Lightweight architectures were developed for mobile use. MobileNet introduced depthwise separable convolutions, and MobileNetV2 added inverted residual blocks and linear bottlenecks.
- Research on dataset size and variety showed that increasing the number of training images does not improve generalization when the added images lack diversity.
- Research gap: existing studies evaluate complete architectures rather than isolating the contribution of individual architectural principles under the same experimental conditions.

Methodology

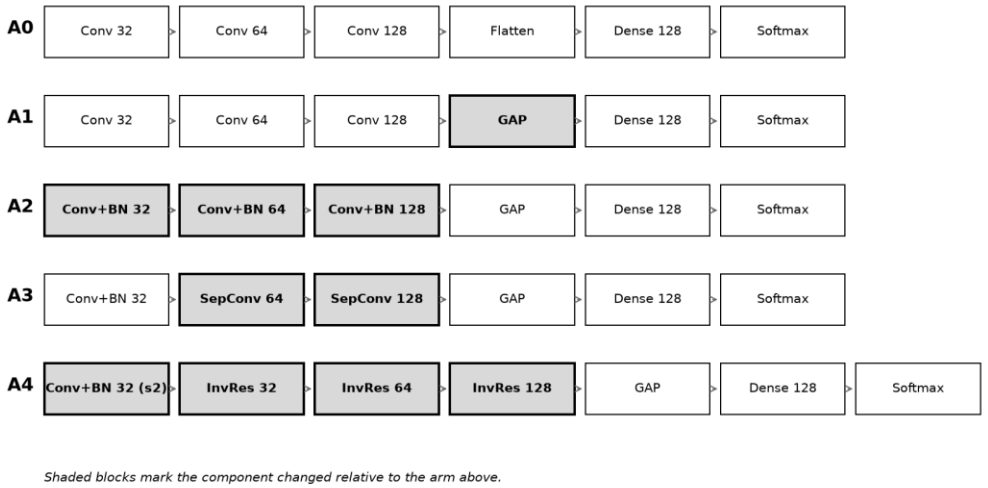
- The dataset was prepared and partitioned to prevent data leakage.
- The images were preprocessed and augmentation was applied to the training set.
- Six model configurations were trained independently under identical settings.
- The trained models were evaluated using validation and held-out test sets.
- Weights from the epoch with the highest validation accuracy were selected.
- The models were compared on classification performance and computational cost.
- Each model differs from the one before it by exactly one change, so the difference between two neighbouring



Dataset and Preprocessing

- The New Plant Diseases Dataset was obtained from Kaggle. It contains 87,220 leaf images from 14 crops across 38 classes, including 12 healthy and 26 diseased classes.
- The crops include apple, blueberry, cherry, maize, grape, orange, peach, bell pepper, potato, raspberry, soybean, squash, strawberry and tomato.
- The dataset also contains augmented copies of the same original photographs, so grouping by source identifier gives only 47,032 distinct photographs.
- In the published split, 8,599 photographs were shared between the training and validation sets, so 62.7% of the validation images were augmented copies of photographs already present in training.
- To prevent this leakage, images were grouped by source identifier before splitting, class proportions were maintained and a fixed seed was used, so no source identifier appears in more than one split.
- The splits are 11,400 training files from 9,559 source images, 4,574 validation images and 9,166 test images. Validation and test contain only original images, while augmented images were retained in the training set.
- All images were resized to 128 x 128 pixels and labels were one-hot encoded. Augmentation was a horizontal flip with rotation and zoom of up to ten per cent, applied to the training set only.

Model Architectures



Shaded blocks mark the component changed relative to the arm above.

- A0 is the baseline custom CNN with three convolution and max-pooling blocks of 32, 64 and 128 filters, a flatten layer, a 128-unit dense layer with 0.5 dropout and a 38-class softmax output.
- A1 replaces the flatten layer with global average pooling, A2 adds batch normalization after each convolution, A3 replaces the second and third convolutions with depthwise separable convolutions, and A4 uses inverted residual blocks with linear bottlenecks.
- B0 is a MobileNetV2 pretrained on ImageNet with its convolutional base frozen and the same classification head. Only 7.0% of its parameters are trainable.

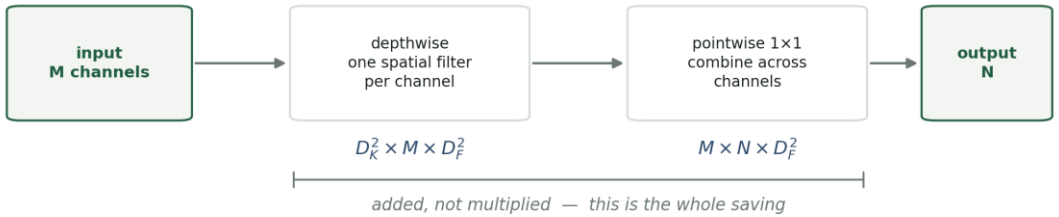
Convolutional Neural Networks

- A CNN learns features from an image through a sequence of layers.
- Convolutional layers apply learnable filters to detect patterns such as edges, textures, shapes and colour variations:
  - $Y(i, j) = \Sigma m \, \Sigma n \, X(i + m, j + n) \, K(m, n)$
- An activation function introduces non-linearity. The Rectified Linear Unit is $f(x) = \max(0, x)$, so negative values become zero and positive values are retained.
- Pooling reduces the spatial dimensions of the feature maps. In max pooling the maximum value within a local region R is selected:
  - $Y(i, j) = \max_{(m, n) \in R} X(i + m, j + n)$
- Simple patterns are combined into more complex features through successive layers, and the classification layers use these features to assign the image to a class.

Algorithm Details

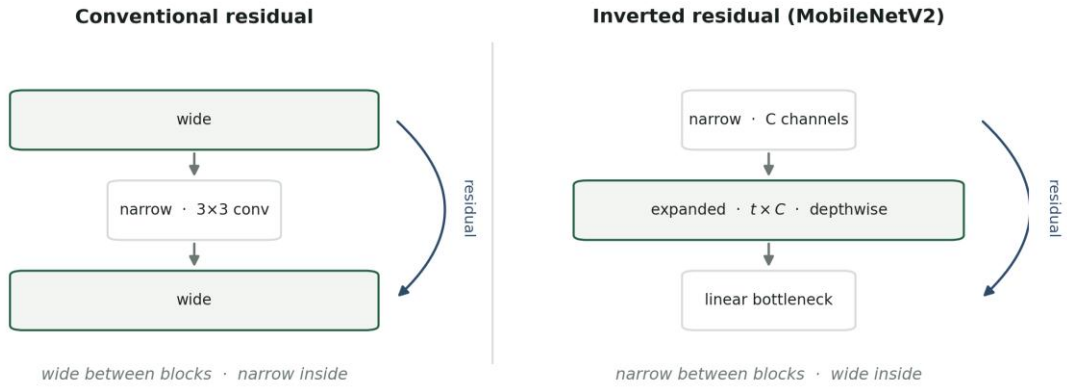
Depthwise Separable Convolution

Standard convolution $D_K^2 \times M \times N \times D_F^2$ filters spatially and mixes channels in one operation



Efficiency ratio $(D_K^2 MN) / (D_K^2 M + MN)$

Inverted Residual Block



Batch Normalization and Global Average Pooling

Batch Normalization

- For a mini-batch of m values the mean and variance are calculated, each activation is normalized, and learned scale and shift parameters are applied:
  - $\mu_B = (1 / m) \Sigma x_i \qquad \sigma_B^2 = (1 / m) \Sigma (x_i - \mu_B)^2$
  - $\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma_B^2 + \varepsilon)} \qquad y_i = \gamma \hat{x}_i + \beta$
- It stabilizes the distribution of activations during training and can improve the convergence of deep networks.

Global Average Pooling

- Instead of flattening all spatial values into a large fully connected layer, the average of each feature map is calculated. For a map of height H and width W:
  - $y_k = (1 / HW) \Sigma_i \Sigma_j x_{i, j, k}$
- The resulting vector contains one value per channel, which greatly reduces the number of parameters required by the classification layer.

Evaluation Metrics

- Classification quality was measured on the held-out test split:
  - Accuracy = correct predictions / total predictions
  - Precision = TP / (TP + FP) Recall = TP / (TP + FN)
  - F1 = 2 · Precision · Recall / (Precision + Recall)
- Precision, recall and F1 were calculated using macro averaging across all 38 classes. Macro F1 was the main metric because it gives equal importance to each class. Accuracy and weighted F1 were also reported.
- McNemar's exact test was used to compare the models, since all six were evaluated on the same 9,166 test images.
- Efficiency was compared using total and trainable parameters, model size, FLOPs, inference latency, throughput and training time.
- Latency was measured on a CPU over 100 runs after a warm-up, recording the median and the 90th percentile. Throughput was measured over ten timed batches of 32.

Implementation Details

- Models were trained on a Kaggle Notebook using a Tesla T4 GPU.
- Inference latency was measured on a CPU for all models.
- Libraries used: TensorFlow, Keras, NumPy, scikit-learn, SciPy and Matplotlib.
- Adam optimizer with a learning rate of 0.001 and categorical cross-entropy loss.
- Batch size of 64 and a fixed budget of 40 epochs, with no early stopping.
- Dropout of 0.5 in the five custom models and 0.3 in the MobileNetV2 reference.
- Random seeds fixed at 42 for Python, NumPy and TensorFlow.
- Weights from the epoch with the highest validation accuracy

```
import tensorflow as tf
gpus = tf.config.list_physical_devices('GPU')
print('TensorFlow', tf.__version__, '| GPUs:', gpus)
assert gpus, 'No GPU attached. Pick a GPU runtime in the kernel picker, then rerun.'

OUTPUT:
TensorFlow 2.20.0 | GPUs:
  [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'),
   PhysicalDevice(name='/physical_device:GPU:1', device_type='GPU')]

From the training log of the same run:
  Created device /job:localhost/replica:0/task:0/device:GPU:0
      with 13756 MB memory:  -> device: 0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5
  Created device /job:localhost/replica:0/task:0/device:GPU:1
      with 13756 MB memory:  -> device: 1, name: Tesla T4, pci bus id: 0000:00:05.0, compute capability: 7.5
```

```
timer = EpochTimer()
callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        str(weights_path), monitor="val_accuracy", mode="max",
        save_best_only=True, save_weights_only=True, verbose=0),
    tf.keras.callbacks.CSVLogger(str(run_dir / "history.csv")),
    timer,
]

start = time.perf_counter()
history = model.fit(train_ds, validation_data=val_ds,
                    epochs=args.epochs, callbacks=callbacks, verbose=2)
train_seconds = time.perf_counter() - start

val_acc = history.history["val_accuracy"]
best_epoch = int(np.argmax(val_acc)) + 1
model.load_weights(weights_path)

probs = model.predict(test_ds, verbose=0)
y_pred = np.argmax(probs, axis=1)
y_true = np.concatenate([np.argmax(y.numpy(), axis=1) for _, y in test_ds])
assert len(y_true) == len(y_pred), f"{len(y_true)} labels vs {len(y_pred)} predictions"

report = classification_report(
```

Implementation Code

```python
def build_custom_cnn(input_shape=(224, 224, 3), num_classes=38):
    model = Sequential([
        Input(shape=input_shape),
        Rescaling(1. / 255),

        Conv2D(32, (3, 3), activation='relu'),
        MaxPooling2D((2, 2)),

        Conv2D(64, (3, 3), activation='relu'),
        MaxPooling2D((2, 2)),

        Conv2D(128, (3, 3), activation='relu'),
        MaxPooling2D((2, 2)),

        Flatten(),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(num_classes, activation='softmax')
    ])
    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

if __name__ == "__main__":
    model = build_custom_cnn()
    model.summary()
```

Baseline A0 constructor

```python
def _inverted_residual(x, filters, expansion=4, stride=1):
    in_channels = x.shape[-1]
    y = Conv2D(in_channels * expansion, (1, 1), padding="same", use_bias=False)(x)
    y = BatchNormalization()(y)
    y = ReLU(max_value=6.0)(y)

    y = DepthwiseConv2D((3, 3), strides=stride, padding="same", use_bias=False)(y)
    y = BatchNormalization()(y)
    y = ReLU(max_value=6.0)(y)

    y = Conv2D(filters, (1, 1), padding="same", use_bias=False)(y)
    y = BatchNormalization()(y)

    if stride == 1 and in_channels == filters:
        y = Add()([x, y])
    return y


def build_a4(input_shape=(224, 224, 3), num_classes=38, expansion=4):
    inputs = Input(shape=input_shape)
    x = Rescaling(1. / 255)(inputs)

    x = Conv2D(CHANNELS[0], (3, 3), strides=2, padding="same", use_bias=False)(x)
    x = BatchNormalization()(x)
    x = ReLU(max_value=6.0)(x)

    for stage, filters in enumerate(CHANNELS):
        x = _inverted_residual(x, filters, expansion=expansion,
                               stride=1 if stage == 0 else 2)
        x = _inverted_residual(x, filters, expansion=expansion, stride=1)

    return _compile(Model(inputs, _head(x, num_classes), name="A4_inverted_residual"))
```

Inverted residual block and A4

```python
def make_dataset(split, img_size, batch_size, seed, classes=None, augment=False, shuffle=False):
    classes = classes or load_classes()
    paths, labels = _read_manifest(split, classes)
    num_classes = len(classes)

    if shuffle:
        order = list(range(len(paths)))
        random.Random(seed).shuffle(order)
        paths = [paths[i] for i in order]
        labels = [labels[i] for i in order]

    ds = tf.data.Dataset.from_tensor_slices((paths, labels))
    ds = ds.map(lambda p, l: _decode(p, l, img_size, num_classes), num_parallel_calls=AUTOTUNE)

    if _cache_bytes(len(paths), img_size) <= CACHE_LIMIT_BYTES:
        ds = ds.cache()

    if shuffle:
        ds = ds.shuffle(min(len(paths), 4096), seed=seed, reshuffle_each_iteration=True)

    ds = ds.batch(batch_size)
    ds = ds.map(lambda x, y: (tf.cast(x, tf.float32), y), num_parallel_calls=AUTOTUNE)
    if augment:
        augmenter = build_augmenter(seed)
        ds = ds.map(lambda x, y: (augmenter(x, training=True), y), num_parallel_calls=AUTOTUNE)
    return ds.prefetch(AUTOTUNE), len(paths)
```

Data pipeline

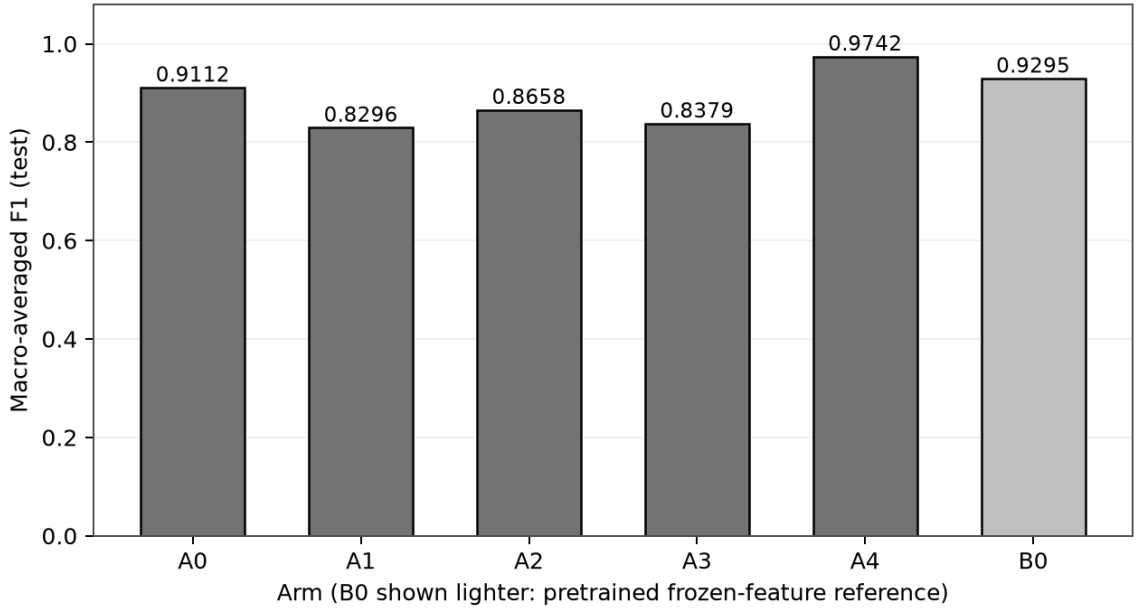
```python
def assign(groups, group_class, val_frac, test_frac, seed):
    by_class = defaultdict(list)
    for uid in groups:
        by_class[group_class[uid]].append(uid)

    rng = random.Random(seed)
    assignment = {}
    for class_name in sorted(by_class):
        uids = sorted(by_class[class_name])
        rng.shuffle(uids)
        n = len(uids)
        n_test = max(1, round(n * test_frac))
        n_val = max(1, round(n * val_frac))
        for uid in uids[:n_test]:
            assignment[uid] = "test"
        for uid in uids[n_test:n_test + n_val]:
            assignment[uid] = "val"
        for uid in uids[n_test + n_val:]:
            assignment[uid] = "train"
    return assignment


def build_rows(groups, group_class, assignment, cap_per_class, seed):
    rows = {"train": [], "val": [], "test": []}
    for uid, paths in groups.items():
        split = assignment[uid]
```

Source-grouped splitting

Results



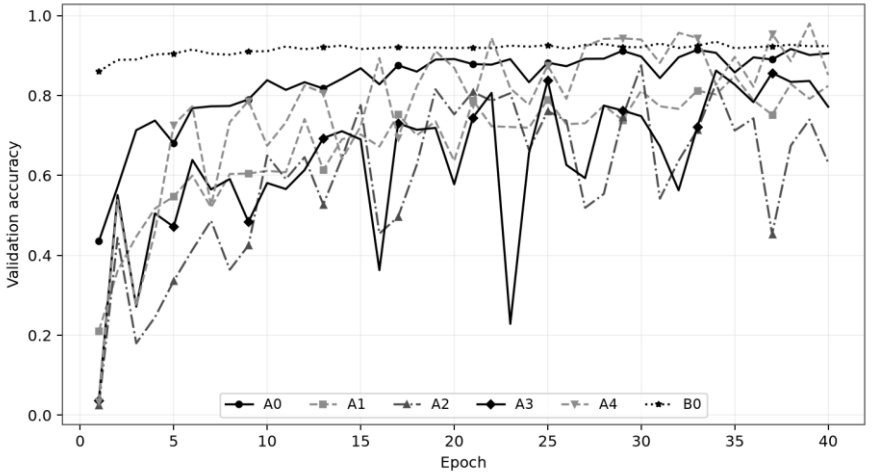

```
arm   macro-F1     acc      params      MB      ms    s/ep     gap
------------------------------------------------------------------
A0      0.9112  0.9210   3,309,542   37.92   23.55      34  -0.031
A1      0.8296  0.8464     114,662    1.36   19.44      34  -0.055
A2      0.8658  0.8750     115,334    1.39   26.20      34   0.033
A3      0.8379  0.8608      34,278    0.47   26.13      34   0.030
A4      0.9742  0.9789     289,798    3.52  109.03      42  -0.002
B0      0.9295  0.9330   2,426,854   10.90  237.56      35   0.007
vs A0 (McNemar exact, discordant pairs A0-only/arm-only):
  A1: macro-F1 -0.0816  params x0.035  p=5.18e-89  (959/275)
  A2: macro-F1 -0.0454  params x0.035  p=2.99e-33  (837/415)
  A3: macro-F1 -0.0733  params x0.010  p=3.92e-52  (953/401)
  A4: macro-F1 +0.0630  params x0.088  p=4.19e-94  (103/634)
  B0: macro-F1 +0.0183  params x0.733  p=7.70e-04  (471/581)
summary -> D:\NN lab 3 assignment\plant disease prediction\output\summary_seed42.csv
figure  -> D:\NN lab 3 assignment\plant disease prediction\scripts\report_figures\ablation_tradeoff.png
```

- All six models were evaluated on 9,166 held-out test images across 38 classes.
- Macro F1 was used as the main metric because the test set keeps its natural class imbalance and macro averaging gives equal weight to every class.
- A4 obtained the highest macro F1-score of 0.9742 and accuracy of 97.89%.
- A4 performed better than the MobileNetV2 reference B0, which obtained 0.9295, and better than the baseline A0, which obtained 0.9112.
- A1, A2 and A3 performed below the baseline, so the changes were not cumulative.
- Macro and weighted F1 differ by at most 0.0252 for any model, so no model gains on the common classes at the expense of the rare ones.

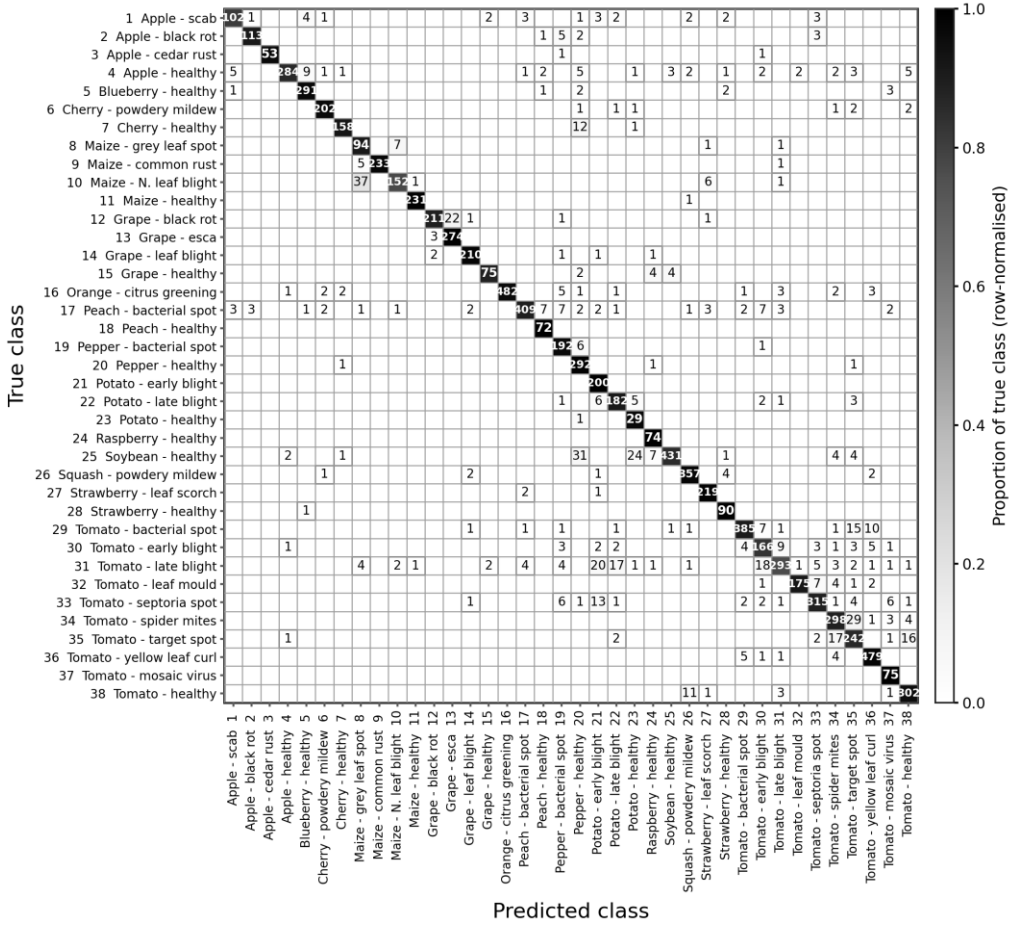
Ablation Results

- Global average pooling reduced macro F1 by 0.0816.
- Batch normalization recovered 0.0362 for only 672 extra parameters.
- Depthwise separable convolutions reduced macro F1 by a further 0.0279, but cut parameters by 3.36 times and gave the smallest model at 0.47 MB.
- Inverted residual blocks contributed 0.1363, about ten times larger than any other effect measured.
- Against the baseline A4 gained 0.0630, and against the reference A4 gained 0.0447.
- McNemar's exact test was used, since all models were evaluated on the same 9,166 test images. Every comparison was statistically significant.
- Two of the four principles reduced accuracy on their own, so the principles are not cumulative and are not equally useful.

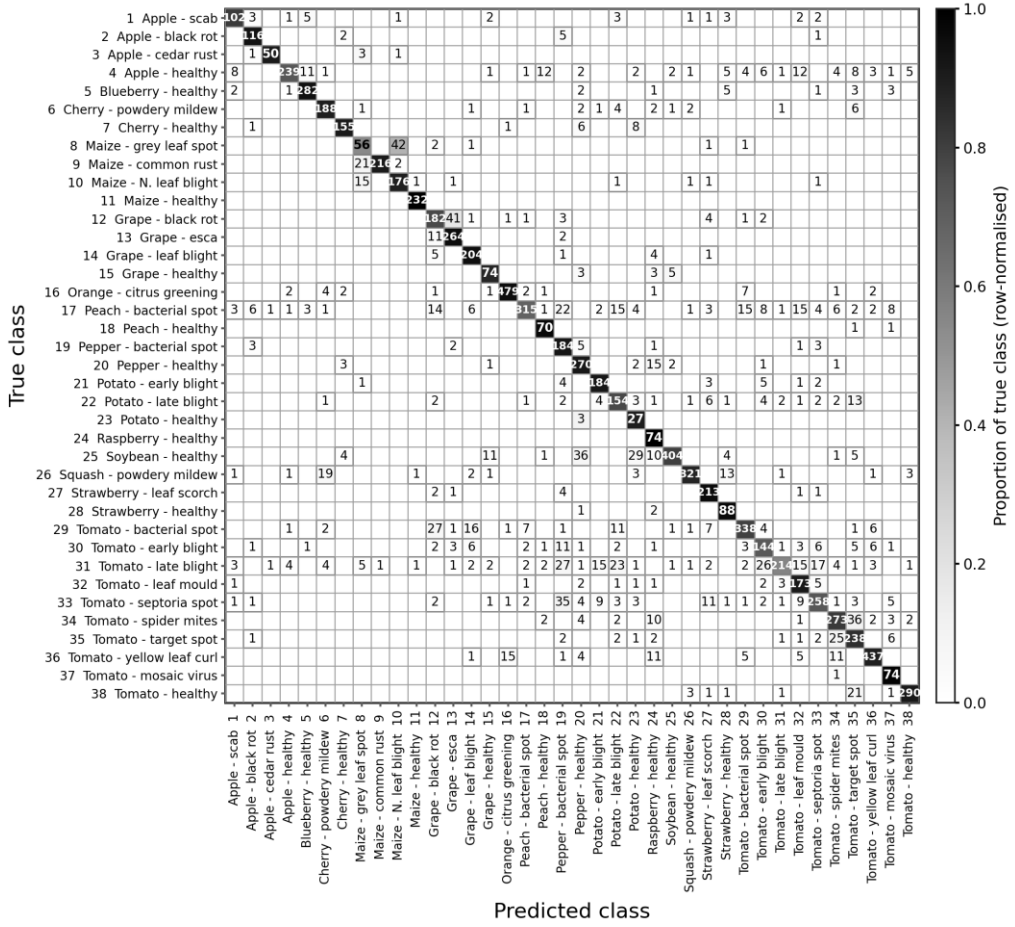
```
arm   macro-F1     acc      params      MB      ms    s/ep     gap
-------------------------------------------------------------------
A0      0.9112  0.9210   3,309,542   37.92   23.55      34  -0.031
A1      0.8296  0.8464     114,662    1.36   19.44      34  -0.055
A2      0.8658  0.8750     115,334    1.39   26.20      34   0.033
A3      0.8379  0.8608      34,278    0.47   26.13      34   0.030
A4      0.9742  0.9789     289,798    3.52  109.03      42  -0.002
B0      0.9295  0.9330   2,426,854   10.90  237.56      35   0.007
vs A0 (McNemar exact, discordant pairs A0-only/arm-only):
  A1: macro-F1 -0.0816  params x0.035  p=5.18e-89  (959/275)
  A2: macro-F1 -0.0454  params x0.035  p=2.99e-33  (837/415)
  A3: macro-F1 -0.0733  params x0.010  p=3.92e-52  (953/401)
  A4: macro-F1 +0.0630  params x0.088  p=4.19e-94  (103/634)
  B0: macro-F1 +0.0183  params x0.733  p=7.70e-04  (471/581)
summary -> D:\NN lab 3 assignment\plant disease prediction\output\summary_seed42.csv
figure  -> D:\NN lab 3 assignment\plant disease prediction\scripts\report_figures\ablation_tradeoff.png
```



Confusion Matrix (1 of 3)

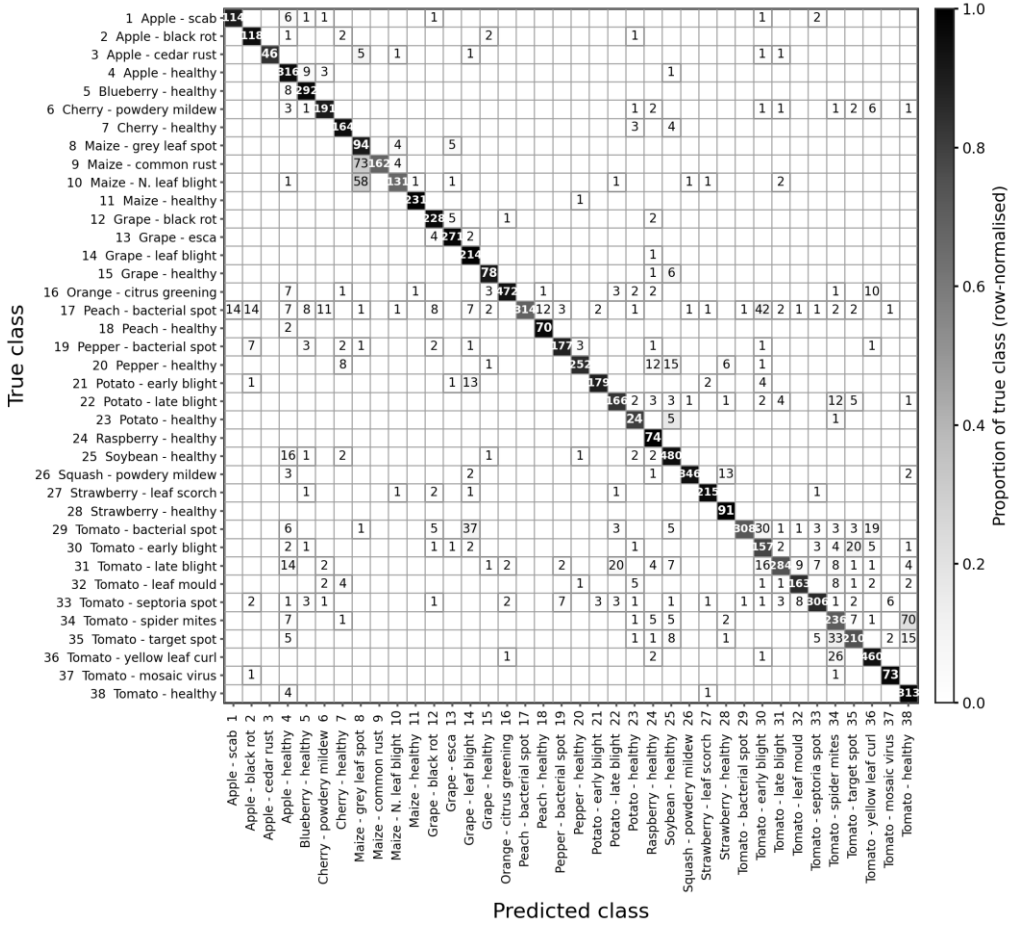


A0 - macro F1 0.9112

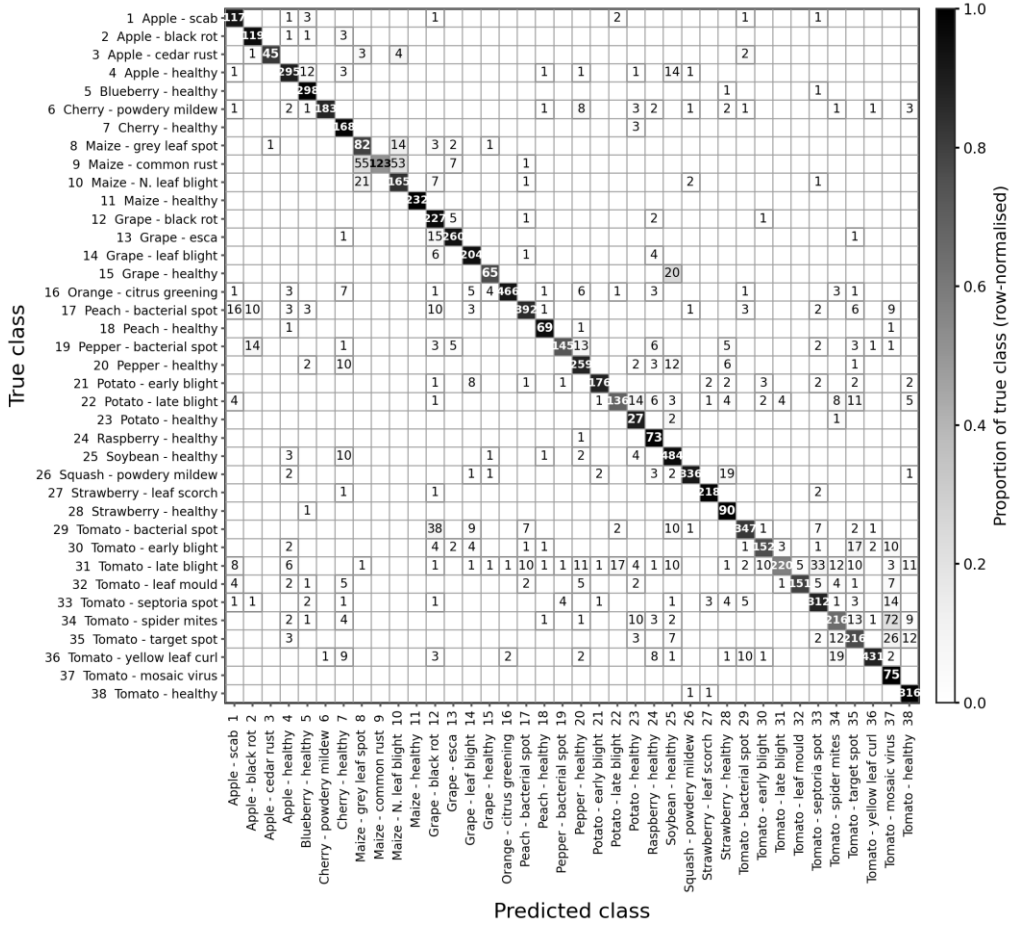


A1 - macro F1 0.8296

Confusion Matrix (2 of 3)

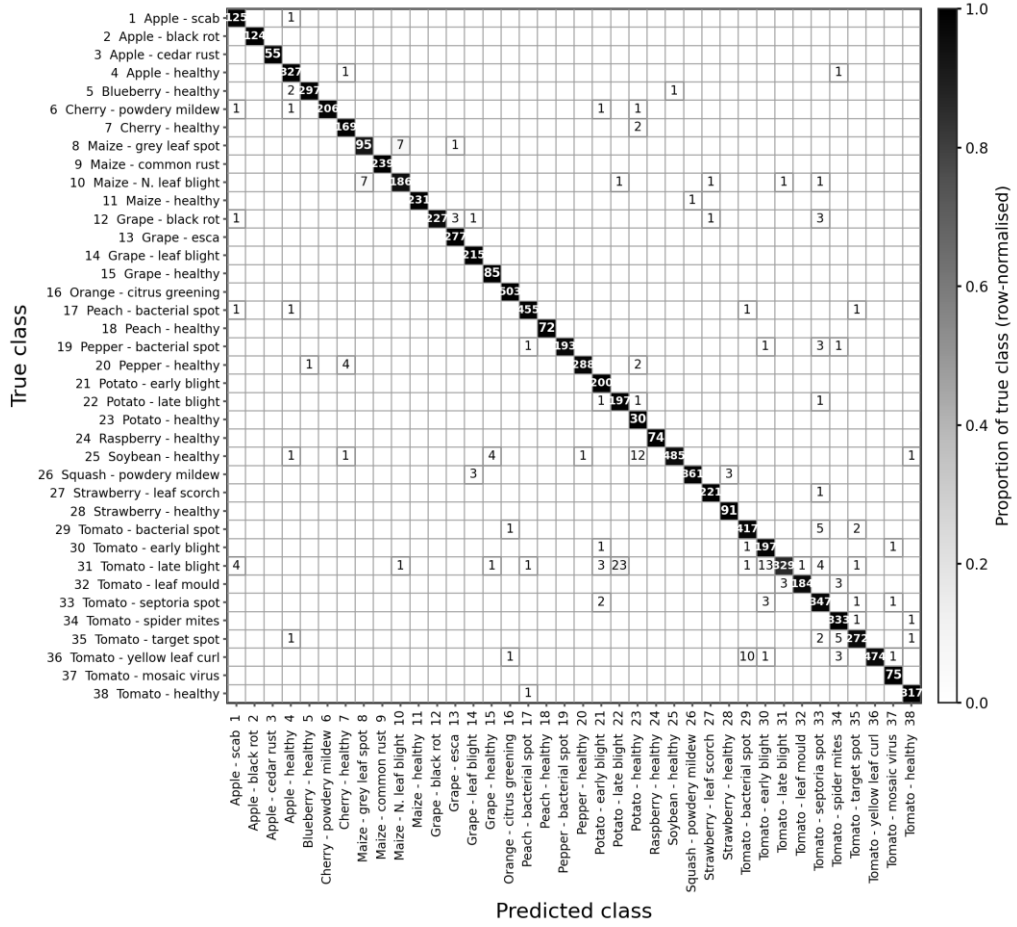


A2 - macro F1 0.8658

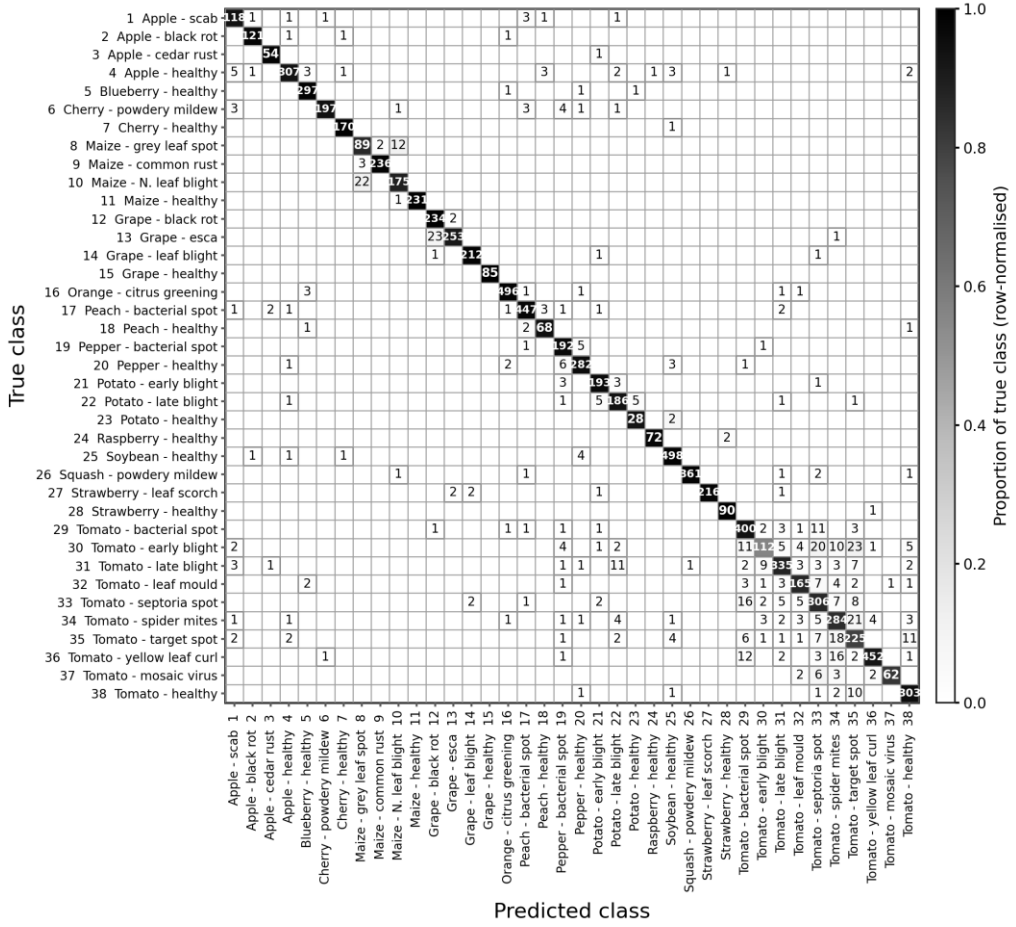


A3 - macro F1 0.8379

Confusion Matrix (3 of 3)



A4 - macro F1 0.9742

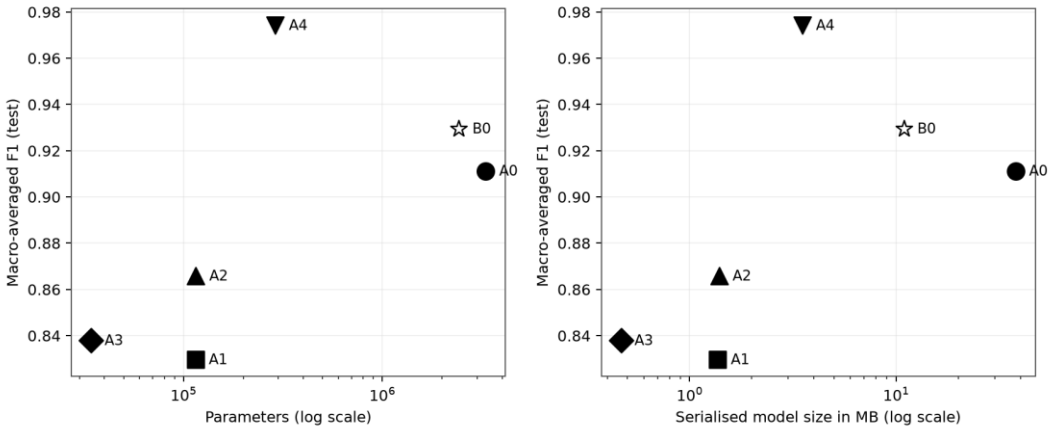


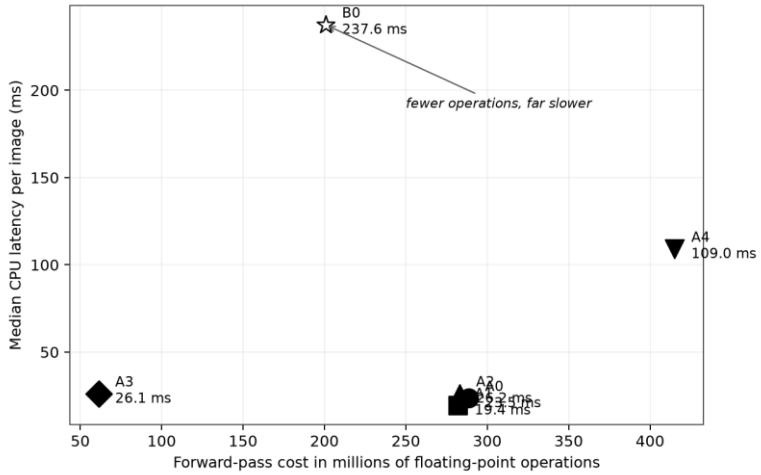
B0 - macro F1 0.9295

Per-Class Analysis

- The test split keeps the original class imbalance, so the classes were divided into thirds by test-set size and per-class F1 was averaged within each third.
  - Smallest third, 12 classes: A0 0.889 A4 0.963 B0 0.932
  - Middle third, 12 classes: A0 0.926 A4 0.979 B0 0.931
  - Largest third, 14 classes: A0 0.917 A4 0.979 B0 0.926
- A4 leads the reference in all three groups, by 0.031, 0.048 and 0.054.
- The reference beats the baseline by 0.043 on the smallest third but by only 0.009 on the largest, which fits the idea that pretrained features substitute for missing data.
- Among the five smallest classes, A4 scores 1.000 on cedar apple rust, peach healthy and raspberry healthy.
- Potato healthy, with only 30 test photographs, is the one class where the reference is clearly ahead, 0.875 against 0.769. With so few images the score is coarse and no conclusion is drawn from it.

Efficiency Analysis





- A4 uses 8.4 times fewer parameters than B0, requires 3.1 times less storage and is 2.18 times faster on the CPU.
- The smallest model is A3 at 0.47 MB, which is 82 times smaller than the baseline.
- B0 performs 1.44 times fewer operations than A0 but takes 10.09 times longer per image, 237.56 against 23.55 milliseconds.
- A4 performs 2.07 times more arithmetic than B0 and still runs 2.18 times faster.
- Time per million operations varies by a factor of 17.2 across the six models.
- Depthwise convolutions reuse little of each value they load, so memory bandwidth rather than arithmetic sets their speed.
- FLOPs are therefore not a reliable proxy for inference speed, and latency should be measured on the target hardware.

Conclusion

- Six models were trained under identical settings and evaluated on 9,166 held-out test images across 38 classes.
- The custom model A4 obtained a macro F1-score of 0.9742, higher than the frozen MobileNetV2 reference at 0.9295, an improvement of 0.0447 that is statistically significant.
- A4 used 8.4 times fewer parameters, required 3.1 times less storage and provided 2.18 times faster CPU inference than the reference.
- The MobileNetV2 principles were not equally useful. Global average pooling and depthwise separable convolutions reduced accuracy individually, while inverted residual blocks were responsible for the improvement.
- FLOPs did not correlate with CPU inference speed, so latency should be measured on the target hardware.
- Selective adoption of MobileNetV2 principles therefore produced a custom CNN with a good balance of accuracy and efficiency.

Future Work

- MobileNetV2 can be fully fine-tuned instead of used as a frozen feature extractor, to provide a stronger reference for comparison with A4.

- The models can be trained for more epochs using multiple random seeds to study training stability and obtain more reliable estimates.

- A learning-rate schedule can be used, since A4 was still improving near the end of the training budget.

- Knowledge distillation can be explored using a stronger teacher model.

- The models can be tested under real deployment conditions on mobile and edge devices, using field images with different backgrounds and lighting.

Limitations: the reference was a frozen feature extractor with only 7.0% trainable parameters, training used a single seed, and the dataset contains single leaves on plain backgrounds rather than field images.

Thank You

Any Questions?

Kriti Mandal | MSc.CSIT, 1st Semester

Central Department of Computer Science and Information Technology